Explainable Rule-Based Detection of Credential Attacks in Authentication Logs

Ndumiso Mbanjwa | CyberFox Society Cybersecurity R&D Internship, Security Operations & Detection Engineering | 30 October 2026 |
https://github.com/onewithalgo-web/cyberfox-detection-engine

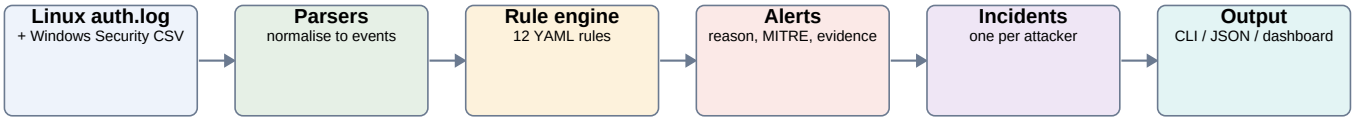
1. Problem

Credential attacks (MITRE ATT&CK T1110 brute force and spraying, T1078 valid accounts) remain one of the most common ways in. Small teams often have no SIEM, and generic thresholds are opaque, noisy and easy to evade by going slow or spreading across IPs. The aim was a **lightweight, explainable, read-only** engine that reads raw authentication logs, alerts on these attacks, shows *why* each alert fired with the log lines as evidence, and is evaluated honestly rather than only on easy data.

2. Methodology

Design principles: every rule is a short YAML file (transparent and tunable from the command line); every alert states its reason, MITRE technique and raw evidence; analysis is read-only and uses only synthetic data plus one public research dataset.
Development loop: build rules; test on labelled synthetic logs that include benign noise (typos, a shared office NAT IP, a night-shift user, VPN source changes) and evasion attacks; study what is missed; add or tune rules; then validate on an **independent real log**.
Scoring: an alert is correct if it has the attack's source, an expected rule and falls in its time window. Because several rules can corroborate one attack, I report *alert precision* and a stricter *incident precision* (one credit per detected attack). For the real log, which has no labels, ground truth was counted separately from the engine directly from the raw text: any IP with 5 or more failed logins is an attacker (a coarse criterion).

3. Implementation



Python, one dependency (PyYAML). Parsers cover Linux sshd logs (including syslog 'message repeated N times' compression) and Windows Security event CSV exports (4624, 4625, 4720, 4732). The engine applies per-key sliding windows (per IP, per account, per IP+account). **Incident grouping** merges corroborating alerts into one incident per attacker. The dashboard is one self-contained HTML file with severity, per-rule, per-IP and per-day views, an incidents table, and click-through raw evidence. Thresholds can be overridden per run (*--set R001.threshold=8*). 33 automated tests pass, including a check that identical events as a Linux log and as a Windows CSV give identical alerts.

ID	Detects (MITRE)	Default
R001 / R007 / R011	Brute force, per IP+account: fast, slow, very slow (T1110.001)	5/2 min, 8/1 h, 10/24 h
R002 / R008	Password spraying: fast and low-and-slow (T1110.003)	4 users/5 min, 5/1 h
R010	Distributed brute force: one account, many IPs (T1110.001)	10 fails, 4+ IPs, 15 min
R012	Probing of non-existent accounts, per IP (T1110.001)	5 / 24 h
R004	Login succeeded after repeated failures, critical (T1110)	3 fails / 10 min
R003 / R005	Off-hours login; login from never-seen source (T1078)	06:00-20:00; 3-day baseline
R006 / R009	Added to privileged group; new account (T1136.001)	any

4. Outcome

Synthetic, labelled (7-day logs, 11 attacks each, 30 seeds):

Step (30 seeds, 330 labelled attacks)	Detection	Alert prec.	Incident prec.	FP / log
A. Week 2 rules (R001-R008)	72.7%	91.7%	86.3%	1.3
B. + R010 distributed brute force	81.8%	92.2%	87.7%	1.3
C. + R011 very slow brute force	90.9%	93.4%	88.8%	1.3
C'. Final rules (R001-R012), no VPN option	90.9%	94.3%	88.8%	1.3
D. + R005 ignore_cidrs (generator's VPN range)	90.9%	99.2%	98.4%	0.2
E. D with 2x / 3x benign noise	90.9%	97.8% / 98.0%	95.5% / 95.9%	0.5 / 0.4

Detection rose from 72.7% to 90.9% through new rules driven by the missed-attack analysis. **Honest reading:** almost all of the precision gain (94.3% to 99.2%) comes from one option, *ignore_cidrs*, which I set to the VPN range my generator uses; an operator would have to know their own network. The remaining misses are the insider with valid credentials (0 of 30).

Independent real-world log (public LogHub OpenSSH data: 528 failed logins from 12 attacker IPs, plus one successful login I treated as legitimate; it raised no alert):

Independent real log (LogHub OpenSSH, 2,000 lines)	Attacker IPs found	Alerts	Alerts on other IPs
First run (R001-R011)	9 / 12	30	0
After parser fixes and R012	12 / 12	39 (= 12 incidents)	0

The 9/12 first run is the clean figure. R012 was written after seeing the miss, so 12/12 is not a held-out result. Grouping reduced triage load from 39 alerts to 12 incidents (one per attacker IP) on the real log and from 635 to 305 on the synthetic set.

Missed-attack analysis (what each miss taught, and the fix):

Missed attack	Found in	Cause	Fix
One account hit from many IPs	Synthetic	Every rule keyed on one IP	R010: correlate by account
1 try / 10 min for 2 h	Synthetic	Windows of 1 h or less	R011: 24 h window
syslog 'message repeated 5 times' (2 attackers)	Real log	Parser counted 1 failure, not 6	Parser expands repeats
3 invalid accounts, 5 tries over 3 h	Real log	Below every rate threshold	R012 (designed after the miss)
Insider with valid credentials, usual IP and hours	Synthetic	Identical to legitimate use	None possible from auth logs

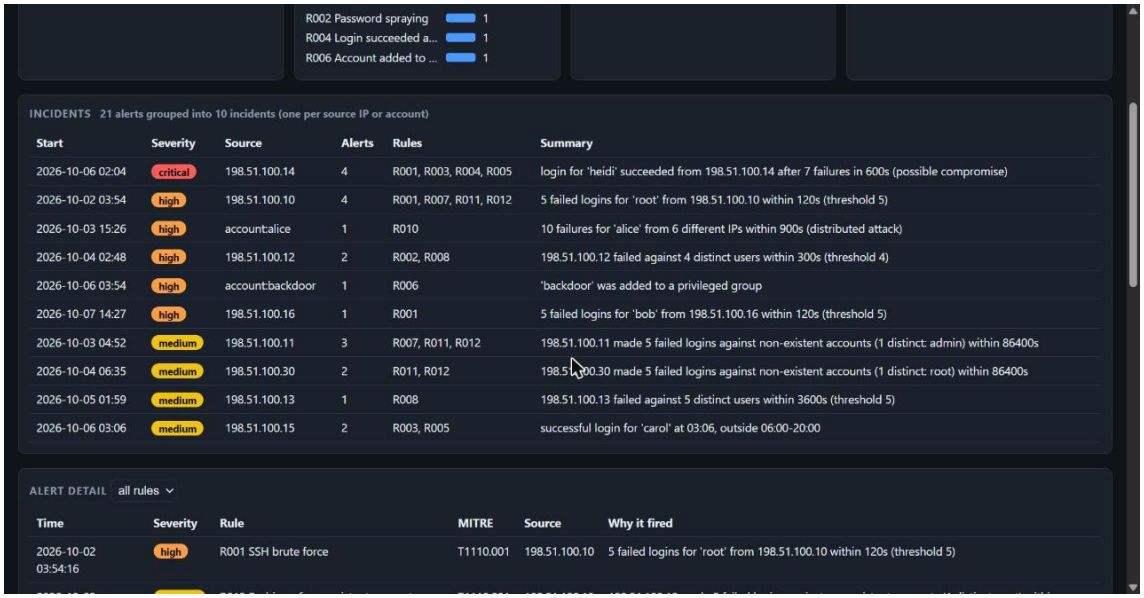


Figure 1. Dashboard incidents panel on the synthetic sample: 21 alerts grouped into 10 incidents, one per attack, with severity, rules fired and the most serious reason. Clicking a row lists its alerts; clicking an alert shows the raw log lines.

5. Limitations, lessons and future work

The rules and options were tuned on the same generator, so synthetic figures show the method works, not production performance; the real test is small (one server, four hours, one attack style). The Windows parser is verified only by a synthetic round-trip. Attacks spread across many accounts and IPs, and misuse of valid credentials, are out of reach of authentication logs alone. Independent testing was the most valuable step: real data exposed two parser/rule gaps that synthetic data could not, and an audit of my own results corrected several claims (including attributing most precision gains to one tuning option). **Future work:** per-user baselines, more real datasets (especially Windows), endpoint telemetry for insider misuse, and a streaming mode.